

编译成功的测试样例

test_success_1.txt — 斐波那契数列 (Fibonacci)

预期运行结果 (输入 n=10) :

```
0
1
1
2
3
5
8
13
21
34
```

说明: 经典 Fibonacci 数列。

test_success_2.txt — 素数判定 (Prime Detection)

预期运行结果:

```
1
0
```

说明: 分别检测 29 (素数) 和 25 (非素数) 。

test_success_3.txt — 欧几里得算法求最大公约数 (GCD)

预期运行结果:

```
48
18
6
1071
462
21
```

说明: $\text{GCD}(48,18)=6$, $\text{GCD}(1071,462)=21$ 。

test_success_4.txt — 集合综合操作

预期运行结果:

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7, 8}
{1, 2, 3, 4, 5, 6, 7, 8}
{4, 5}
```

```
{1, 2, 3, 4, 5, 10}
{1, 2, 3, 4, 5, 10}
{1, 2, 3, 4, 5}
{1, 2, 4, 5}
{1, 2, 4, 5}
1
0
false
true
{100, 200, 300}
```

说明： `add` 不破坏互异性，`remove` 对不存在元素无操作。测试了括号内算术表达式 `(6*2-7) in s1`。

test_success_5.txt — 嵌套块变量遮蔽

预期运行结果：

```
10
{1, 2}
true
20
{5, 6}
{1, 2}
true
{5, 6, 7}
30
{5, 6, 7}
{2, 10}
20
{5, 6, 7}
{2, 10}
10
{2, 10}
false
0
```

说明： 三层嵌套变量遮蔽。

test_success_6.txt — 埃拉托斯特尼筛法 (Sieve of Eratosthenes)

用集合 `primes` 存素数、`composites` 存合数。while 循环遍历 2~30，用 `in` 判断是否已标记为合数，若非则输出并将倍数逐一 `add` 到 `composites`。最后用 `inter/union/==/isempty` 验证素数集与合数集不相交。

预期运行结果（输入 n=30 后自动计算）：

```
2
3
5
7
11
13
17
```

```
19
23
29
{2, 3, 5, 7, 11, 13, 17, 19, 23, 29}
{4, 6, 8, 9, 10, 12, 14, 15, 16, 18, 20, 21, 22, 24, 25, 26, 27, 28, 30}
true
true
false
```

说明：前 10 行为逐个素数，第 11 行为素数集，第 12 行为合数集。`primes inter composites` 为空 → `isempty` 为 true。`primes union composites` 是全 2~30，不等于 `primes` → false。

test_success_7.txt — 对称差集与集合恒等式验证

手算对称差集 $(a \cup b) - (a \cap b)$ ：while 循环遍历 1~20，对每个数统计在 `a` 和 `b` 中的归属 (`x = 0/1/2`)，若恰好只在一个集合中 (`x == 1`) 则 add 到 `diff`。最后用 3 个集合恒等式验证结果。

预期运行结果：

```
{2, 4, 6, 8, 10}
{1, 2, 3, 5, 8, 13}
{1, 3, 4, 5, 6, 10, 13}
true
true
true
```

说明：第 3 行为对称差集。3 个 true 分别验证：(1) `a inter b == {2,8}` 交集精确匹配；(2) `a inter b != {}` 交集非空；(3) `(a union b) == (b union a)` 并集交换律。

test_success_bonus_2.txt — 集合相等比较（加分项 2）

预期运行结果：

```
true
false
true
false
true
false
true
true
true
false
true
```

说明：集合相等比较忽略元素顺序。空集之间相等。

test_success_bonus_3.txt — 集合推导式（加分项 3，参考用）

```
{ set s1; set s2;
  s1 = {1,2,3,4,5}; write s1;
  s2 = { x + 1 | x in s1 if x > 2 }; write s2;
  s1 = {10,20,30,40,50};
  s2 = { x / 2 | x in s1 if x > 25 }; write s2; }
```

预期运行结果：

```
{1, 2, 3, 4, 5}
{4, 5, 6}
{15, 20, 25}
```

编译失败的测试样例

test_fail_1.txt — 声明缺少分号

```
{ int a      ← 缺少 ;
  int b; a = 1; b = 2 }
```

预期错误：缺少分号。解析器在 `int a` 后期望 `;`，却读到 `int`。

test_fail_2.txt — 缺少 }

```
{ int x; x = 10; write x;  ← 没有 }
```

预期错误：缺少 `}`。block 解析完语句后期望 `}` 却读到 EOF。

test_fail_3.txt — 未声明变量

```
{ int x; x = 10; write y; } ← y 未声明
```

预期错误：标识符未声明。`write y` 中的 `y` 在符号表中找不到。

test_fail_4.txt — 类型不匹配

```
{ int x; bool b; x = 10; b = true; x = b; } ← bool 赋值给 int
```

预期错误：类型不匹配。`b` 是 `bool` 类型，不能赋值给 `int` 类型的 `x`。

test_fail_5.txt — isempty 参数非集合

```
{ int x; set s; bool b; x = 10; s = {1,2,3}; b = isempty(x); }
```

预期错误: isempty 需要 set 类型参数。x 是 int。

test_fail_6.txt — 同作用域重复声明

```
{ int x; int x; x = 10; write x; } ← x 声明两次
```

预期错误: 变量重复声明。同一作用域内不能声明同名变量。

test_fail_7.txt — if 缺少 (

```
{ int x; x = 10; if x > 5 { write x; } } ← 缺少 ( )
```

预期错误: if 后应有 (。L26 语法要求 if (bexpr)。

test_fail_8.txt — read 不能读集合

```
{ set s; s = {1,2,3}; read s; } ← read 不支持 set 类型
```

预期错误: 不能读取到 set 类型变量。

test_fail_9.txt — 非法字符

```
{ int x; x = 10; @@@; write x; } ← @@@ 不合法
```

预期错误: 不可识别的符号。@ 不在 L26 的字符集中。

test_fail_10.txt — 声明语法错误

```
{ set s,y; ← 不支持一次声明多个变量  
  s = {1,2,3}; y = {1,2,3}; }
```

预期错误: 一次声明多个变量。

test_fail_11.txt — 集合字面量缺少 }

```
{ set s; s = {1, 2, 3; } ← 缺少 }, 分号被当作元素分隔
```

预期错误: 集合字面量缺少 }。解析逗号分隔的元素时遇到 ;。

test_fail_12.txt — read 后跟数字

```
{ int x; read 123; } ← read 后应为标识符
```

预期错误：read 后应为标识符。`123` 是数字不是变量名。

test_fail_13.txt — in 右侧不是集合

```
{ int x; set s; bool b; x = 5; s = {1,2,3};  
  if(true) { int s; s=10; b = x in s; } } ← x 不是 set
```

预期错误：in 右侧需要 set 类型。

test_fail_14.txt — 标识符命名错误

```
{ int 1x; int y; y=10;} ← 1x 命名不规范
```

预期错误：不允许命名为1x。

test_fail_15.txt — if 条件误用 =

```
{ int x; int y; x=10; y=10;  
  if(x=10) { y=5; } } ← 应该用 == 而不是 =
```

预期错误：`x=10` 被解析为赋值表达式（`=` 不是关系运算符）。